T09 # Hygiène du Workspace

Technique Beginner

Garder le contexte propre en bloquant ce que Claude ne doit pas lire

Il n'y a pas de .claudeignore

Claude Code n'a pas de fichier `.claudeignore` à la `.gitignore`. Le blocage de fichiers passe par `permissions.deny` dans `.claude/settings.json`, avec des patterns de chemins directs tels que `Read(*.env*)`. Le qualificatif `file_path:` est invalide pour les règles de permission Read, Edit et Write.

```
{
  "permissions": {
    "deny": [
      "Read(*.env*)",
      "Edit(*.env*)",
      "Write(*.env*)",
      "Read(*.pem)",
      "Read(*.key)",
      "Read(*credentials*)",
      "Bash(cat .env*)"
    ]
  }
}
```

Pourquoi c'est important

`deny` a la priorité la plus haute : aucune règle `allow` ou `ask`, à aucun niveau, ne peut la contourner. La règle bloque les appels `Read`, `Edit`, `Write`, et les commandes Bash qui correspondent explicitement à un pattern deny. Elle ne bloque pas les commandes Bash génériques sans règle deny dédiée, ni un `ls .env*` qui révèle l'existence du fichier sans son contenu.

Ce que permissions.deny couvre

Vecteur	Bloqué
`Read()`	Oui
`Edit()` / `Write()` avec règle	Oui
`Bash(cat .env)` avec règle explicite	Oui
Bash générique sans règle deny	Non
`ls .env*` (nom de fichier)	Partiel

Pour une protection en profondeur : stocker les secrets hors du répertoire projet, ajouter un hook PreToolUse en second verrou, ne jamais committer de secrets même « bloqués ».

.gitignore vs permissions.deny

Mécanisme	Rôle	Scope
`.gitignore`	Exclut du contrôle de version	Git
`permissions.deny`	Bloque la lecture/édition/écriture par les outils	Claude Code
`respectGitignore` (défaut : `true`)	Le picker `@` respecte les patterns `.gitignore`	Claude Code

Ces trois mécanismes sont indépendants. Un fichier peut être ignoré par Git sans être bloqué pour Claude, et inversement.

Structure recommandée du dossier .claude/

```
.claude/
├── settings.json        # Config équipe (commité)
├── settings.local.json  # Config locale (gitignore)
├── agents/              # Agents personnalisés
├── skills/              # Skills (inclut les commandes slash depuis CC 2.1.3)
├── hooks/               # Scripts d'automatisation
└── rules/               # Conventions partagées, éventuellement limitées à des chemins
```

Les dossiers `agents/`, `skills/`, `hooks/` et `rules/` se committent avec le projet. `settings.local.json` et `CLAUDE.local.md` à la racine du projet contiennent la configuration locale et les instructions personnelles. La mémoire automatique réside sous `~/.claude/projects/<project>/memory/`, séparément de cet arbre.

Checklist workspace propre

- `.env*`, `*.pem`, `*.key`, `*credentials*` dans `permissions.deny`
- Règles deny couvrant Read, Edit, Write ET les commandes Bash équivalentes
- Secrets stockés hors du répertoire projet quand possible
- `settings.local.json` dans `.gitignore`
- Hook PreToolUse en second verrou sur les fichiers critiques